

Appunti di Basi di Dati

Nicholas Scorrano

A.A 2025/2026

Capitolo 4

SQL

4.1 Cosa è SQL?

SQL - Structured Query Language è il linguaggio standard utilizzato per la definizione e la manipolazione dei dati nei database relazionali

Le funzionalità di SQL si dividono in 2 categorie principali

1. DDL - Data Definition Language

È la componente del linguaggio che permette di creare e modificare la struttura del database.

Include le operazioni per definire schemi, tabelle, domini e vincoli.

2. DML - Data Manipulation Language

È la componente dedicata all'interrogazione all'aggiornamento dei dati veri e propri all'interno delle tabelle

4.2 Caratteristiche di SQL

- **È un linguaggio dichiarativo**

In SQL l'utente non descrive come il sistema debba estrarre o modificare i dati passo dopo passo.

Non potendo scegliere l'ordine in cui avvengono le operazioni a livello di sistema, è sufficiente attenersi rigidamente alla struttura sintattica delle istruzioni per descrivere il risultato desiderato

- **È relazionalmente completo**

Questa caratteristica indica che ogni singola espressione concepibile nell'algebra relazionale può essere tradotta in una corrispondente istruzione SQL

- **Adotta una logica a 3 valori**

Per poter gestire l'informazione incompleta e in particolare il valore NULL, SQL non si limita alla logica booleana classica, ma utilizza 3 sintattica

1. **T** : True / Vero
2. **F** : False / Falso
3. **U** : Unknown / Sconosciuto

4.3 Notazione

La notazione convenzionale utilizzata per descrivere la sintassi e leggere i comandi SQL segue queste specifiche regole

- **Termini in maiuscolo**

Indicano le parole chiave e i termini che appartengono al linguaggio SQL

- **Termini in minuscolo**

Rappresentano i termini variabili che devono essere scelti e specificati dall'utente (come ad esempio i nomi da dare alle tabelle o agli attributi)

- **<x> (Parentesi angolari)**

Vengono utilizzate per isolare un determinato termine.

- **[x] (Parentesi quadre)**

Indicano che il termine racchiuso al loro interno è opzionale (può essere inserito o omesso)

- **{x} (Parentesi Graffe)**

Significano che il termine al loro interno può essere ripetuto zero volte oppure un numero arbitrario di volte

- **| (Barra Verticale)**

Serve a separare opzioni alternative

4.4 Istruzioni Principali DDL

4.4.1 CREATE

Serve a definire e creare la struttura degli oggetti nel database.

Viene utilizzata per creare database, tabelle, domini, viste, vincoli e autorizzazioni

Creazione degli Schemi

Uno schema di base di dati è una collezione di oggetti, come domini, tabelle, viste, asserzioni e privilegi.

```
1 CREATE [NomeSchema] [[authorization] Autorizzazione]{
2     DefinizioneElementoSchema
3 }
```

Listing 4.1: questa è la sintassi della creazione di uno schema

```
1 CREATE SCHEMA Didattica AUTHORIZATION coordinatore
```

Listing 4.2: Questo è un esempio pratico di creazione di uno schema

Creazione Tabella

```
1 CREATE TABLE NomeTabella (
2     NomeAttributo Dominio [ValoreDiDefault] [Vincoli], [AltriVincoli]
3 );
```

Listing 4.3: Sintassi della creazione di una tabella

```

1 CREATE TABLE Prodotto (
2     ID INT PRIMARY KEY,
3     Nome VARCHAR(100) NOT NULL,
4     Prezzo DECIMAL(10, 2) CHECK (Prezzo > 0),
5     Categoria VARCHAR(50) DEFAULT 'Varie'
6 );

```

Listing 4.4: Esempio pratico di creazione di una tabella

Una volta preparato lo schema, si creano le tabelle al suo interno.

L'istruzione CREATE TABLE definisce la struttura logica della tabella e ne crea un'istanza inizialmente vuota.

Viene ricordato che in SQL si usa il termine "tabella" e "riga" proprio perchè il linguaggio ammette la presenza di righe duplicate, a differenza del modello matematico teorico

4.4.2 ALTER

Viene utilizzata per modificare la struttura o le proprietà di oggetti già esistenti nel database, come la modifica degli attributi o dei vincoli

ALTER di una tabella

Permette di alterare, aggiungere o rimuovere singole colonne e vincoli all'interno di una tabella

```

1 ALTER TABLE NomeTabella <
2     ALTER COLUMN NomeAttributo < SET DEFAULT NuovoDefault | DROP DEFAULT > |
3     DROP COLUMN NomeAttributo |
4     ADD COLUMN DefAttributo |
5     DROP CONSTRAINT NomeVincolo |
6     ADD CONSTRAINT DefVincolo
7 >

```

Listing 4.5: Sintassi di ALTER TABLE

```

1 ALTER TABLE PRODOTTO
2 ALTER COLUMN prezzo-unitario SET DEFAULT 10;

```

Listing 4.6: Esempio pratico di ALTER TABLE

4.4.3 DROP

È il comando dedicato all'eliminazione permanente di database, tabelle, domini e altri oggetti dallo schema.

```

1 DROP < schema, domain, table, view, ...> NomeElemento [ RESTRICT | CASCADE ]

```

Listing 4.7: Sintassi di DROP

Poichè è un operazione distruttiva, offre 2 opzioni di esecuzione :

- **RESTRICT**

Che impedisce la rimozione se l'oggetto contiene istanze dipendenti non vuote

- **CASCADE**

Che invece procede con una reazione a catena eliminando l'oggetto e tutto ciò che è coinvolto nella sua definizione

4.5 Domini (Tipi di Dato)

In SQL, i domini hanno lo scopo di specificare in modo preciso l'insieme dei valori ammessi per ciascun attributo all'interno di una tabella

Essi si suddividono in 2 categorie principali

4.5.1 Domini elementari (predefiniti)

Sono i tipi di dato nativi messi a disposizione dallo standard SQL.

I principali sono :

- **Testuali (Carattere)**

Rappresentano singoli caratteri alfanumerici o stringhe.

Si distinguono in stringhe a lunghezza fissa (CHAR) e stringhe a lunghezza variabile (VARCHAR), per le quali è possibile specificare la lunghezza massima

- **Numerici Esatti**

Rappresentano numeri interi (come INTEGER)

- **Numerici Approssimati**

Utilizzati spesso per le grandezze fisiche, si basano sulla rappresentazione in virgola mobile

Includono domini come float, double precision, real

- **Data e Ora**

Gestiscono le informazioni temporali tramite domini come TIMESTAMP, DATE, TIME

- **Bit / Boolean**

Il tipo Bit (con valori 0 o 1) presente in SQL-2 è stato parzialmente sostituito da BOOLEAN in SQL-3

- **BLOB e CLOB**

Oltre ai 6 di base, esistono i domini per i grandi oggetti (Binary/Character Large Object), utilizzati per trattare informazioni multimediali o non strutturate come immagini, video e testi lunghi

4.5.2 Domini Definiti dall'Utente

Partendo dai domini predefiniti, è possibile costruire nuovi domini personalizzati e riutilizzabili utilizzando l'istruzione CREATE DOMAIN

```
1 CREATE DOMAIN NomeDominio AS DominioElementare [ ValoreDefault ] [ Constraints ]
```

Listing 4.8: Sintassi per creare un Dominio

Le diverse componenti della sintassi indicano le caratteristiche che avrà il nuovo dominio :

- **NomeDominio** : Definisce il nome personalizzato del dominio
- **DominioElementare** : È il tipo di dato predefinito (interi, stringhe, date, ...) da cui il nuovo dominio prende origine
- **ValoreDefault** : È un parametro opzionale che specifica il valore che verrà inserito automaticamente se non viene specificato alcun dato

- **Constraints (Vincoli)** : È un parametro opzionale che permette di definire regole ferree sui valori accettati, usando vincoli come NOT NULL oppure condizioni logiche tramite la clausola CHECK <

```

1 CREATE DOMAIN Voto AS SMALLINT
2 DEFAULT 0
3 CHECK (value >= 18 AND value <= 30)

```

Listing 4.9: Creazione del dominio "Voto"

4.5.3 Valori DEFAULT e Valore Nullo (NULL)

DEFAULT

Quando si definiscono i domini, è possibile impostare un valore di DEFAULT, che l'attributo assumerà automaticamente se non viene specificato alcun dato in frasi di inserimento

NULL

Il valore NULL è un valore "polimorfico" che appartiene a tutti i domini.

Esso non equivale a zero o a uno spazio vuoto, ma indica esplicitamente che un'informazione è non nota, non applicabile, oppure che non si sa se l'informazione sia applicabile o meno

4.6 Vincoli Intrarelazionali

I vincoli intrarelazionali sono regole o proprietà che devono risultare sempre valide all'interno di una singola relazione (ovvero una singola tabella).

```

1 CREATE TABLE Impiegato (
2     Matricola CHAR(6) PRIMARY KEY,
3     Nome CHAR(20) NOT NULL,
4     Cognome CHAR(20) NOT NULL,
5     Stipendio NUMERIC(9) DEFAULT 0 CONSTRAINT StipendioPositivo CHECK (Stipendio
6     > 0),
7     UNIQUE (Cognome, Nome)
8 );

```

Listing 4.10: Esempio Pratico

In SQL, i vincoli principali che si applicano all'interno della stessa tabella sono 4 :

- **NOT NULL**

Impone che il valore inserito in un attributo debba necessariamente essere non nullo

- **UNIQUE**

Garantisce che i valori in una colonna non si ripetano mai, trasformando gli attributi coinvolti in una superchiave.

C'è però un'eccezione : il valore NULL può comparire su diverse righe senza violare il vincolo, poichè il database considera ogni valore nullo diverso dagli altri

- **PRIMARY KEY**

Serve a definire la chiave primaria della tabella.

Equivale ad applicare simultaneamente i vincoli UNIQUE e NOT NULL e può essere definita una sola volta all'interno di una tabella.

- **CHECK**

È un costrutto introdotto in SQL - 2 che permette di definire condizioni logiche o matematiche specifiche sui valori ammessi.

4.7 Vincoli di Interrelazionali (Integrità Referenziale)

```
1  -- 1. Creazione della Tabella Padre
2  CREATE TABLE Studente (
3      Matr CHAR(6) PRIMARY KEY,
4      Nome VARCHAR(30) NOT NULL,
5      Citta VARCHAR(20)
6  );
7
8  -- 2. Creazione della Tabella Figlio
9  CREATE TABLE Esame (
10     Matr CHAR(6),
11     CodCorso CHAR(6),
12     Data DATE NOT NULL,
13     Voto SMALLINT,
14     PRIMARY KEY (Matr, CodCorso),
15
16 -- Definizione del vincolo interrelazionale (Foreign Key)
17 FOREIGN KEY (Matr) REFERENCES Studente(Matr)
18     ON DELETE CASCADE -- Se elimino lo studente, vengono cancellati a cascata
                           anche i suoi esami
19     ON UPDATE CASCADE -- Se modifico la matricola dello studente, si aggiorna
                           anche negli esami
20 );
```

I vincoli Interrelazionali legano tra loro tabelle diverse, definendo un rapporto gerarchico tra una "Tabella Padre"(o esterna) e una "Tabella Figlio"(o interna).

La regola principale è l'integrità referenziale : i valori inseriti negli attributi chiave della tabella figlio (la cosiddetta Foreign Key / Chiave Esterna) debbono obbligatoriamente comparire come valori chiave primaria della tabella padre

Ci sono modalità per definire questi vincoli

1. Sintassi in linea (per un singolo attributo)

Si dichiara direttamente di fianco all'attributo usando la sintassi

```
1  AttrFiglio TipoDato REFERENCES TabellaPadre(AttrPadre)
```

Listing 4.11: Sintassi in linea

2. Sintassi a fine tabella (per uno o più attributi)

Si usa alla fine del blocco CREATE TABLE

```
1  FOREIGN KEY (AttrFiglio1, AttrFiglio2) REFERENCES TabellaPadre(AttrPadre1,
2      AttrPadre2)
```